

PROJECT UPDATE · 2026.10

hs / simulate

함수형 사고를 실행 가능한 경험으로

개발자용 커리큘럼과 시각적 프로토타입에서
간단한 게임을 만드는 MVP로 확장합니다.

함수를 연결하고, 결과를 보며 함수형 프로그래밍을 익힙니다.

WHO

기성 프로그래머

기존 개발 경험을 바탕으로 함수형 사고와 추상화 이해

HOW

시각적 실습

노드 연결과 실행으로 함수의 입력·출력·합성을 확인

NEXT

게임 제작

학습한 개념으로 작게라도 동작하는 결과물 완성

개념을 아는 것과 직접 만드는 것 사이의 간격

01

합성은 추상적

함수가 다른 함수의 입력이 되는
흐름을 이해하기 어렵습니다.

02

타입은 낮설다

타입 서명과 실제 값의 관계를 실행과
함께 볼 필요가 있습니다.

03

실습의 목표가 필요

작은 예제를 넘어 하나의 결과물을
만드는 경험이 필요합니다.

접근 방식: 개념 학습 → 시각적 실습 → 작은 게임 구현

지금까지: 커리큘럼과 프로토타입 설계

기존 작업

개발자용 함수형 커리큘럼

함수형 프로그래밍을 배우기 위한 학습 흐름과 실습 방향을 설계했습니다.

발표에서는 핵심 개념과 다음 실습 단계만 요약합니다.

구현 확인

시각적 함수 실험 프로토타입

캔버스에서 함수를 연결하고 실행하며 타입을 확인하는 기반을 구현했습니다.

Vite 기반 웹 앱 · JavaScript 실행기 · 자체 타입 추론

현재 성과는 학습 기반 구축. 다음 목표는 게임 제작 가능성 검증입니다.

커리큘럼: 핵심 개념 목록

각 개념을 이해하고, 함수 연결과 구현에 적용합니다.

FOUNDATION

함수 중심 사고

- 입력과 출력
- 순수 함수
- 불변성
- 참조 투명성

COMPOSITION

함수와 추상화

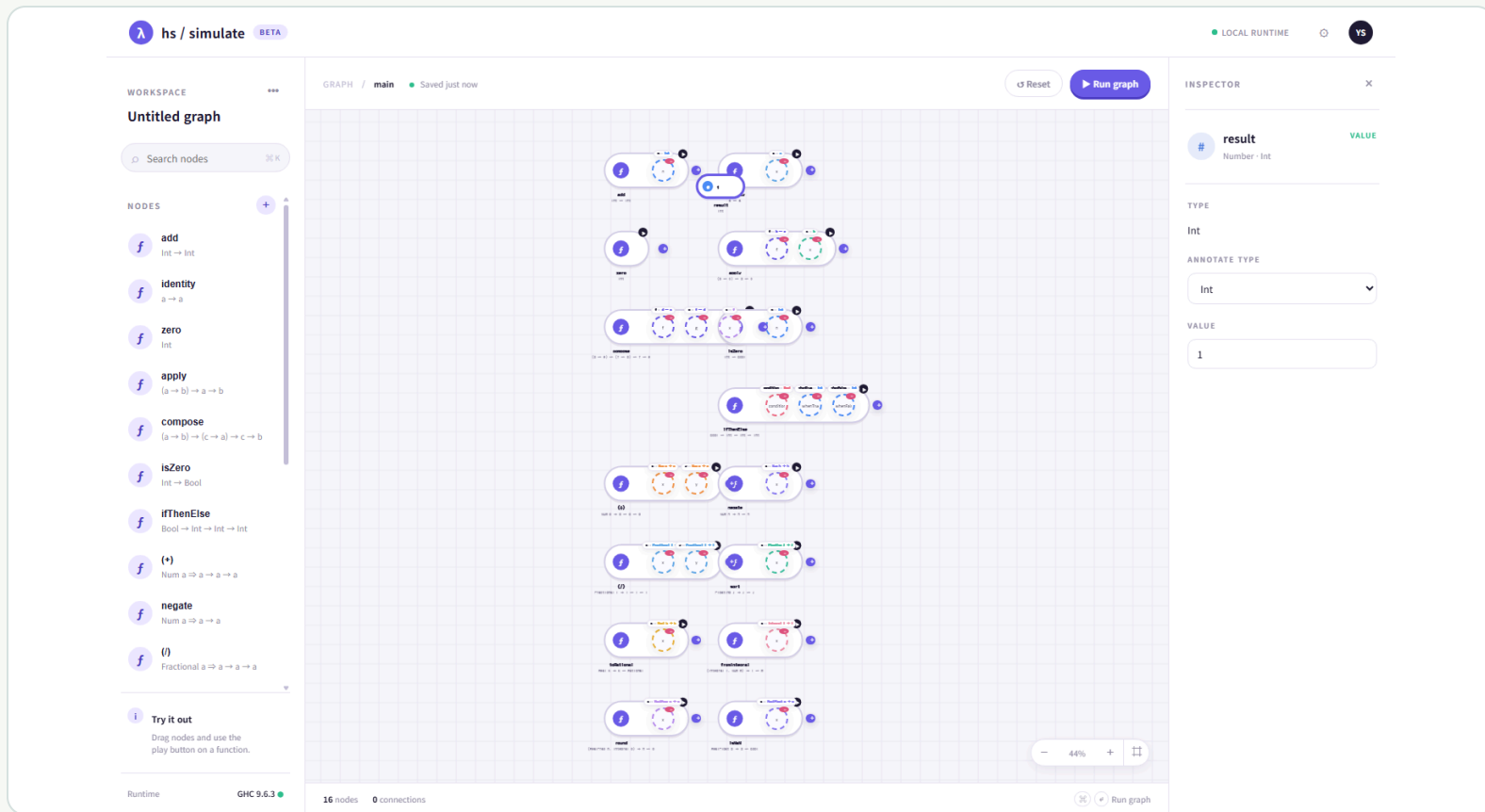
- 일급 함수
- 고차 함수
- 커링과 부분 적용
- 함수 합성

TYPES

타입 시스템

- 타입 서명
- 다형성
- 타입 추론
- 타입클래스

프로토타입: 연결 → 확인 → 실행



함수 선택·연결

캔버스에서 함수의 입력과 출력을 구성합니다.

타입 확인

인스펙터로 타입 서명을 확인합니다.

실행 결과

Run graph로 결과 노드를 생성합니다.

로컬 프로토타입 실제 실행 화면 · 현재 실행기는 JavaScript 기반

현재 구현된 핵심 기능

01 노드 기반 편집 · 함수 본체와 출력 연결

02 커스텀 함수 · 함수 합성 · 부분 적용 표현

03 다형 타입 추론 · 맞지 않는 타입 연결 차단

04 숫자 타입클래스 제약 · 타입 명시와 디폴팅 안내

현재 저장소의 README 및 구현 기준. 완성된 게임 제작 환경은 다음 단계의 목표입니다.

가장 큰 차별점: 함수 자체를 추상화

동작의 묶음을 넘어, 함수의 조합 방식까지 재사용합니다.

스크래치 · 엔트리

블록·함수로 동작 재사용

사용자 정의 블록이나 함수로 반복되는 동작을 묶어 다시 사용합니다.

두 도구에도 추상화와 재사용 기능이 있습니다.

hs / simulate의 핵심

함수를 값으로 조합

함수를 입력으로 전달하고 합성·부분 적용해 새로운 함수를 만듭니다.

다형 타입으로 구체적인 값과 타입을 넘어 일반화합니다.

차별화 방향: “무슨 동작을 묶을까?” → “어떤 함수 조합을 일반화할까?”

기능 참고: [Scratch 사용자 정의 블록](#) · [엔트리 함수 블록](#) / 차별화 설명은 본 프로젝트의 설계 방향

역할 분담: 구현과 검증을 병행

메인 개발

양병선

- MVP 구조와 기능 우선순위 설계
- 게임 입력·상태 갱신·화면 출력 구현
- 함수·타입 시스템 확장 및 코드 통합
- 저장·불러오기와 게임 실행 흐름 완성

테스트 및 검증 · 서버 개발

김종면

- 타입 연결·함수 실행 테스트와 오류 재현
- 예제 게임 제작으로 MVP 동작 검증
- 재시작·저장 복원·사용성 확인
- UI 보완·예제·오류 수정 등 서버 개발

공동 진행: 매주 하스켈 학습 · 구현 리뷰 · 테스트 결과로 다음 작업 결정

다음 단계는 이 프로그램으로 게임을 만드는 것

기성 프로그래머의 함수형 이해를 높이고,
작은 결과물을 완성하는 MVP로.

입력, 상태 변화, 결과 표시를 연결해
간단한 게임의 규칙을 직접 구성합니다.

MVP에는 게임을 완성할 최소 기능만

INPUT

플레이어 입력

버튼 또는 키 입력을 게임 규칙에 전달합니다.

STATE

상태 갱신

점수·턴·진행 상태를 함수로 계산합니다.

OUTPUT

결과 표시

현재 상태와 승패를 화면에 보여줍니다.

추가 목표: 게임 그래프 저장·불러오기, 시작·재시작, 따라 할 수 있는 예제

위 기능은 향후 개발 범위입니다. 게임 규칙은 함수로, 입력·화면 처리는 실행 환경에서 담당합니다.

간단한 게임으로 단계적으로 검증

FIRST · 필수 목표

카운터 목표 게임

버튼 입력으로 점수를 올려 목표에 도달

검증: 입력 → 상태 갱신 → 종료 조건

SECOND · 후속 목표

숫자 맞추기

고정된 정답과 입력을 비교해 힌트 제공

검증: 비교 · 분기 · 시도 횟수
무작위 정답은 이후 확장

OPTIONAL · 여유 시

턴 기반 선택 게임

선택에 따라 점수와 다음 턴을 계산

검증: 규칙 조합 · 함수 재사용

게임 후보와 우선순위 제안. 첫 게임을 완성한 뒤 두 번째 게임으로 재사용성을 확인합니다.

8주 마일스톤 제안

착수 시점 기준 상대 일정 · 구현 결과에 따라 조정

기간	개발 초점	완료 기준
1-2주	실행 흐름 정리 · 첫 게임 설계	입력·상태·출력 구조 확정
3-4주	게임 실행 기반 구현	입력에 따라 상태·화면 갱신
5-6주	첫 게임 완성 · 저장·재시작	그래프 구성으로 한 게임 플레이
7-8주	두 번째 게임 · 사용성 개선	기존 기능 재사용 · MVP 시연

개발과 병행해 매주 하스켈 학습

주차	하스켈 학습 주제	프로젝트 연결
1-2주	순수 함수 · 타입 서명 · 패턴 매칭	게임 규칙을 작은 함수로 분해
3-4주	고차 함수 · 커링 · 함수 합성	노드 조합과 부분 적용 이해
5-6주	대수적 데이터 타입 · Maybe · Either	게임 상태와 오류 표현 설계
7-8주	타입클래스 · IO · 순수/효과 분리	게임 규칙과 실행 환경의 경계 정리

운영 제안: 매주 개념 학습 + 짧은 코드 실습 + 프로젝트 적용점 기록. 앱은 현재 JavaScript 기반입니다.

MVP 완료는 실제 플레이로 판단

01 프로그램 안에서 규칙을 구성한 게임 1개 완성

02 입력 → 상태 변화 → 결과 표시 → 재시작까지 동작

03 저장한 게임을 다시 열어 같은 동작 재현

04 함수 합성·타입·상태 갱신을 예제로 설명 가능

후속 검증: 두 번째 게임에서도 같은 구성 요소를 재사용할 수 있는지 확인합니다.

배운 개념을 연결하고, 직접 만든 게임으로 확인합니다.

지금까지 — 개발자용 커리큘럼과 시각적 프로토타입

앞으로 — 함수형 이해를 돕고, 간단한 게임을 제작하는 MVP

매주 — 하스켈 학습과 프로젝트 적용

다음 행동: 첫 게임 확정 → 입력·상태·출력 설계 → 주별 개발·학습 시작